Overview

Programming basics are not "beginner-only" skills in AI engineering. They are reliability skills. In production AI systems, model quality can fail because of weak data checks, poor control flow, hidden state, or unhandled exceptions.

Python is dominant in AI because it offers: - Fast iteration speed and readable syntax. - Rich ML/data ecosystem (`numpy`, `pandas`, `scikit-learn`, `torch`, `xgboost`). - Easy integration with APIs, data stores, and orchestration systems.

This chapter builds conceptual and practical fluency in Python fundamentals with AI-style examples.

Setting Up Your Environment

Use isolated environments to avoid dependency drift between projects.

1. Create environment:
   - `uv venv --python 3.12.10`
2. Activate environment:
   - `source .venv/bin/activate`
3. Install dependencies:
   - `uv pip install -r requirements.txt`
4. Start notebooks:
   - `jupyter lab`

Environment isolation prevents "works on my machine" failures during team handoff.

Python Basics

Variables and Types

A variable binds name to object in memory. Core built-in types: - Numeric: `int`, `float` - Text: `str` - Logical: `bool` - Collections: `list`, `tuple`, `dict`, `set`

Formal view: if variable x has type T, operations on x should respect domain and codomain constraints of T.

Control Flow

Control flow determines execution path.

- `if/elif/else`: branch on conditions.
- `for`: iterate finite sequences.
- `while`: iterate until predicate fails.

Good AI pipelines use explicit branching for data quality gates, fallback models, and retry logic.

Functions and Reuse

Function maps inputs to outputs with optional side effects.

A pure function can be written as:

$$f : XightarrowY$$

where output depends only on input.

Purity is valuable for testing, reproducibility, and debugging.

Modules and Imports

Modules separate concerns. Example split: - `io_utils.py` for file/API input. - `feature_utils.py` for transforms. - `train.py` for model training.

This reduces notebook sprawl and supports CI test coverage.

Error Handling

Exceptions represent abnormal runtime states.

Use: - `try/except` for recoverable errors. - `raise` for invalid states. - Custom exceptions for domain-specific failures.

Example domains: - `InvalidSchemaError` - `PredictionServiceTimeoutError`

Logging vs Print

`print` is ad hoc. Logging supports levels and traceability: - `DEBUG`: local diagnostics. - `INFO`: expected progress events. - `WARNING`: suspicious but recoverable events. - `ERROR`: operation failed.

In AI operations, logs enable root-cause analysis for data drift, API outages, and model regressions.

Working in Jupyter

Jupyter is ideal for iterative learning and exploratory analysis, but must be disciplined for production readiness.

Best practices: - Keep code cells deterministic and ordered. - Restart kernel + Run All before sharing. - Move stable logic into `.py` modules. - Keep markdown close to code to explain assumptions and caveats.

Think of notebook as lab notebook plus executable spec.

Common Pitfalls & Exceptions

Hidden State

Running cells out of order creates stale variables and false confidence.

Silent Type Coercion

String numbers (`"42"`) and numeric values (`42`) behave differently in arithmetic and sorting.

Broad Exception Handling

`except Exception:` without context can hide real bugs.

Hardcoded Paths

Absolute local paths break in CI or teammate environments.

Mini Projects

Project 1: CLI Metric Calculator

Build command-line utility that validates numeric input and computes mean, variance, and z-score.

Project 2: File Processor

Read CSV, validate required columns, create transformed output, and log row-level failures without crashing entire job.

These small projects mirror common data-engineering tasks in AI teams.

Business Case Studies & Exceptions

Case 1: ETL Preprocessor for Recommendation Features

A batch job reads user events and computes daily aggregates. Failure mode: malformed timestamps cause downstream model features to become null.

Protection pattern: - Strict parsing with explicit exception handling. - Log invalid rows with identifiers. - Continue processing valid rows. - Track error ratio threshold; fail job if threshold exceeded.

Case 2: Online Inference Input Validation

Prediction endpoint receives untrusted payloads. Missing/invalid fields can crash model code or return nonsense scores.

Protection pattern: - Validate input schema and ranges before feature transform. - Raise business exceptions (`ValueError`, custom validation errors). - Return clear HTTP error payload with actionable message.

Interview Questions & Answers

1. **Q: Why is Python preferred in AI engineering?**
   A: Fast prototyping, readable syntax, and mature ecosystem for data processing, modeling, and deployment.
2. **Q: Difference between syntax error and runtime exception?**
   A: Syntax errors occur before execution due to invalid code grammar; runtime exceptions occur during execution on specific inputs or states.
3. **Q: Why use functions instead of writing everything inline?**
   A: Functions improve reuse, testability, and clarity; they isolate logic and reduce duplication.
4. **Q: When should you raise exceptions?**
   A: Raise exceptions when invariants are violated (invalid input, missing required resources, unsafe state).
5. **Q: Logging vs print in production?**
   A: Logging adds levels, timestamps, structured metadata, and centralized collection; print lacks operational observability.
6. **Q: What is module design in Python?**
   A: Organizing related functions/classes into files with clear responsibilities and stable interfaces.
7. **Q: How do you keep notebooks reproducible?**
   A: Ordered deterministic cells, fixed seeds, restart/run-all checks, and moving stable code to modules.
8. **Q: What is defensive programming in AI pipelines?**
   A: Validating assumptions early (schema, types, ranges), handling known failures gracefully, and failing fast on unsafe states.
9. **Q: Give example of bad exception handling.**
   A: Catching all exceptions and returning default values silently, which hides data quality issues.

10. **Q: Why do input/output basics matter for ML engineers?**
    A: Most failures happen at boundaries: file ingestion, API payload parsing, serialization, and schema mismatches.